Data Structure Using C++

Lecture : 4

Doubly Linked Lists & Stack Using Linked List

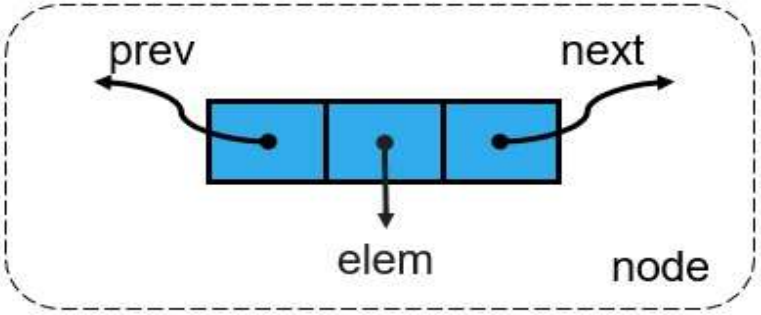
Dr. Ali Hamzah

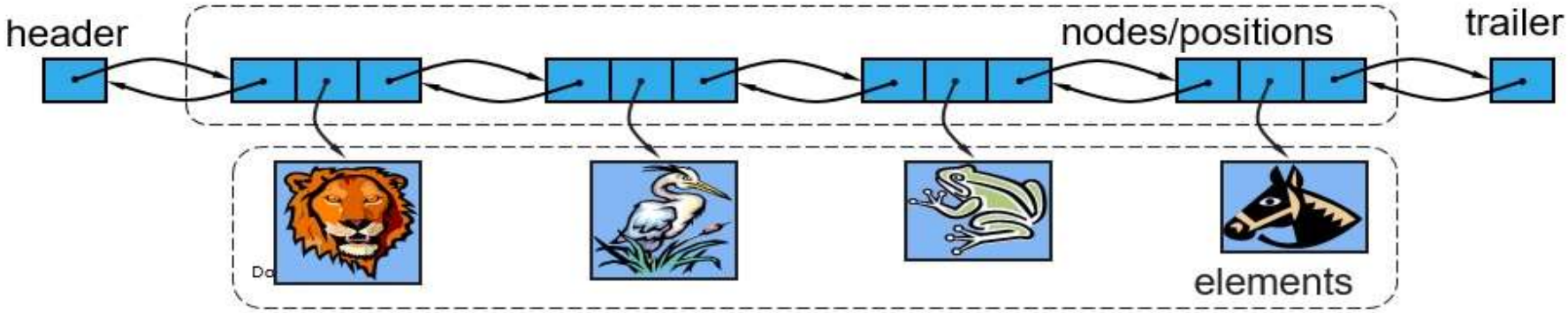
Doubly Linked Lists

Doubly Linked Lists

Doubly Linked List

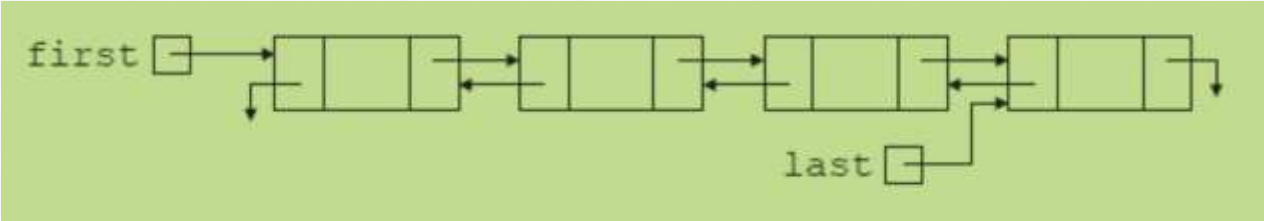
- A doubly linked list provides a natural implementation of the Node List ADT
- Nodes implement Position and store:
  - element
  - link to the previous node
  - link to the next node
- Special trailer and header nodes





Doubly Linked Lists (cont'd.)

- Linked list in which every node has a `next` pointer and a `Pervious` pointer
  - Every node contains address of next node
    - Except last node
  - Every node contains address of previous node
    - Except the first node



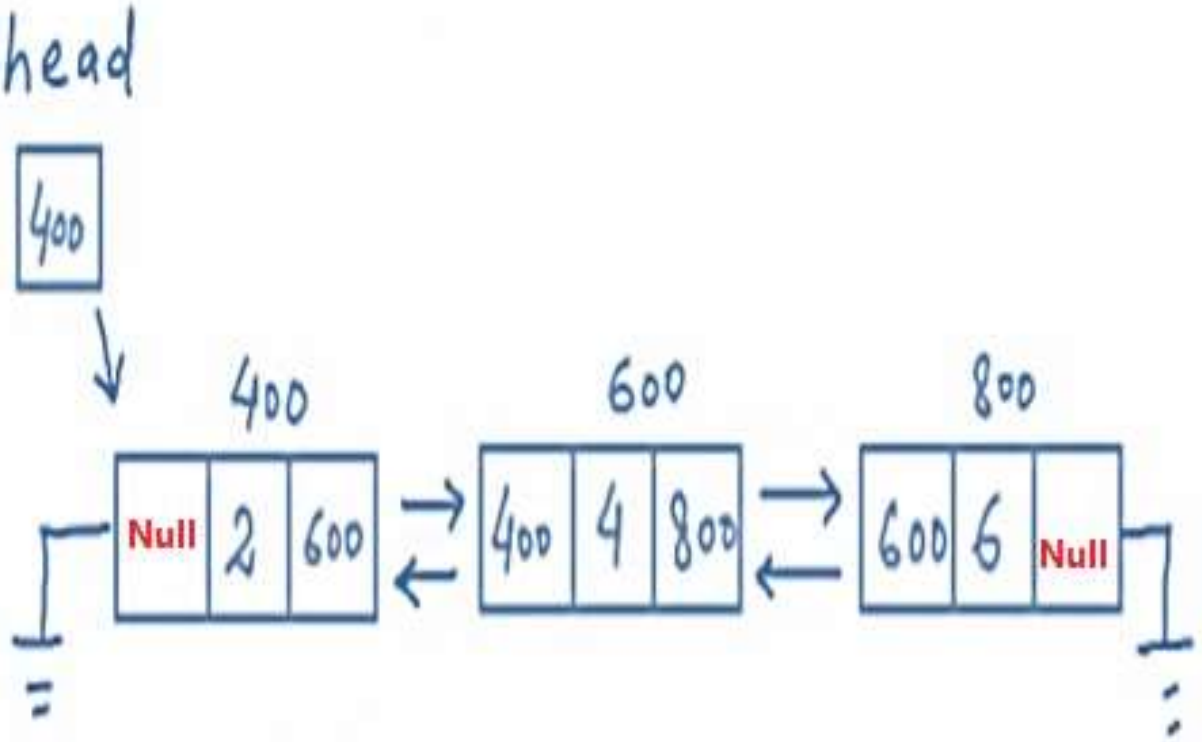
Doubly linked list

Doubly Linked Lists

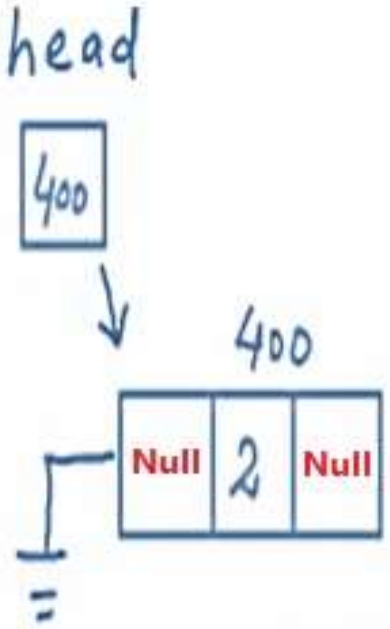
- Traversed in either direction
- Typical operations
  - Initialize the list
  - Determine if list empty
  - Search list for a given item
  - Insert an item
  - Delete an item, and so on

Doubly Linked Lists (cont'd)

```
struct Node  {
    int data;
    struct Node* next;
    struct Node* prev;
};
```



Doubly Linked Lists (Initialize the list)



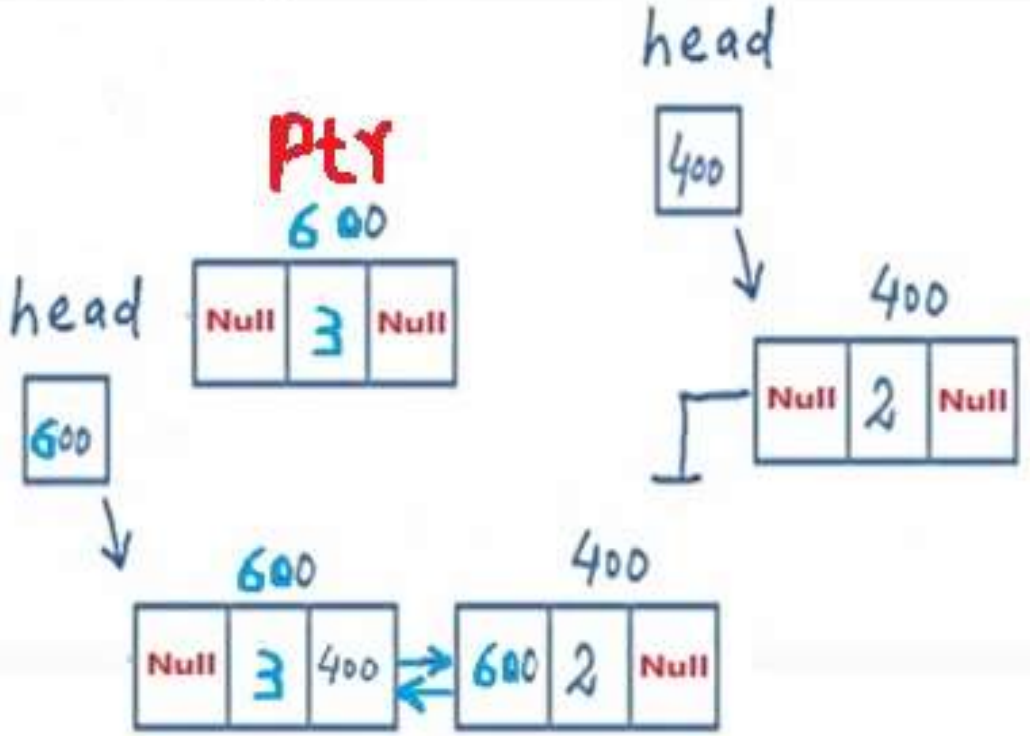

```
struct Node  {
    int data;
    struct Node* next;
    struct Node* prev;
};
```

```
Node *ptr = new node ()
Ptr->data = val;
Ptr->next = val;
Ptr->pre = Null;
```

```
If (Head == Null) {
Head= ptr;
Ptr->Next = Null
Ptr->pr = Null
}
```

Doubly Linked Lists (Insert at front)

```
struct Node  {
    int data;
    struct Node* next;
    struct Node* prev;
};
```

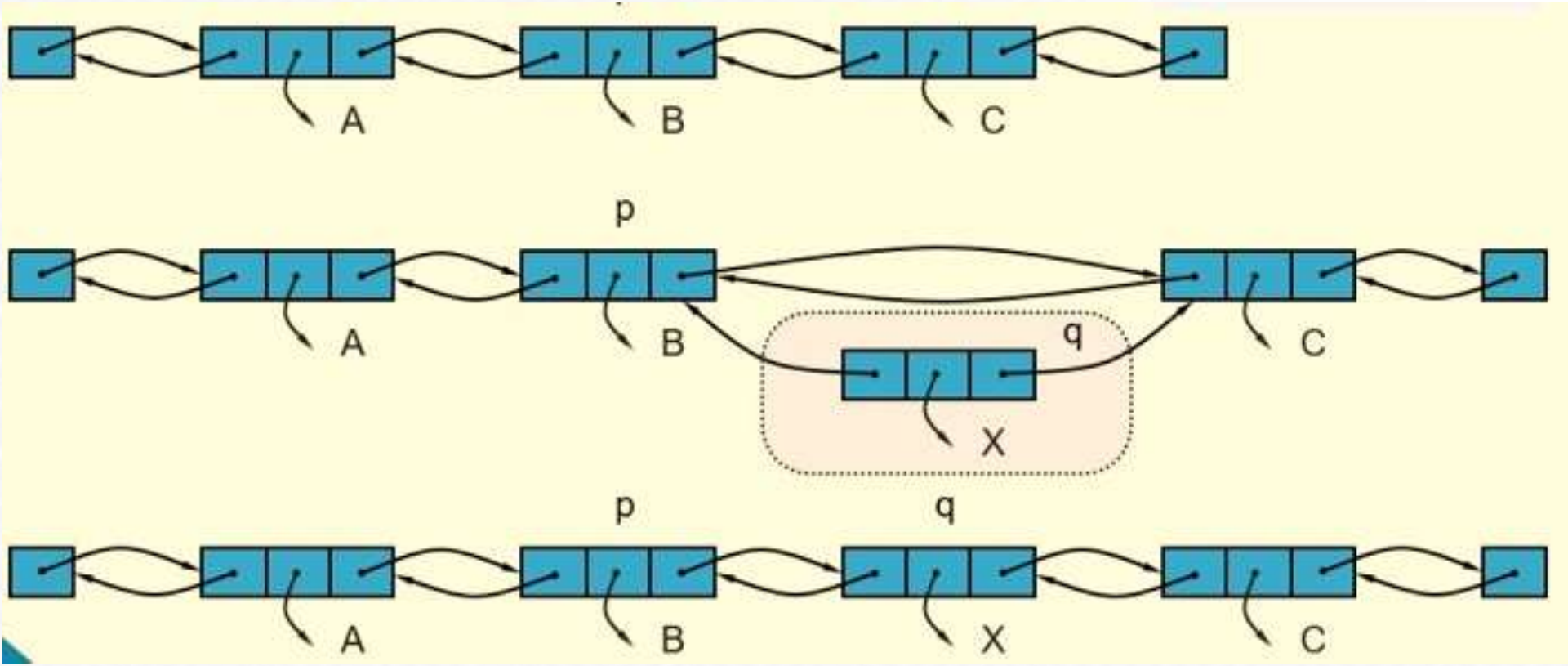


Node *ptr = new node
Ptr->data = val;
Ptr->pre = Null

→

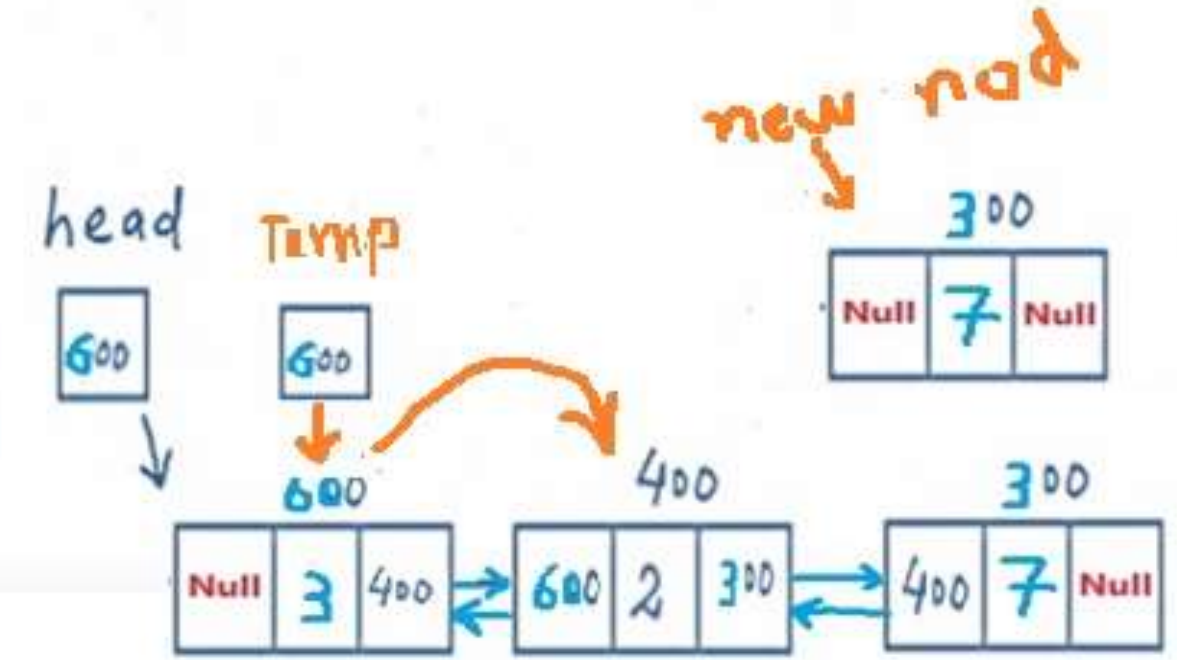
Ptr->Next = Head
Head->pre = ptr
Head= ptr;

Doubly Linked Lists (Insertion in middle)



Doubly Linked Lists (Insert at Last)

```
struct Node  {
    int data;
    struct Node* next;
    struct Node* prev;
};
```

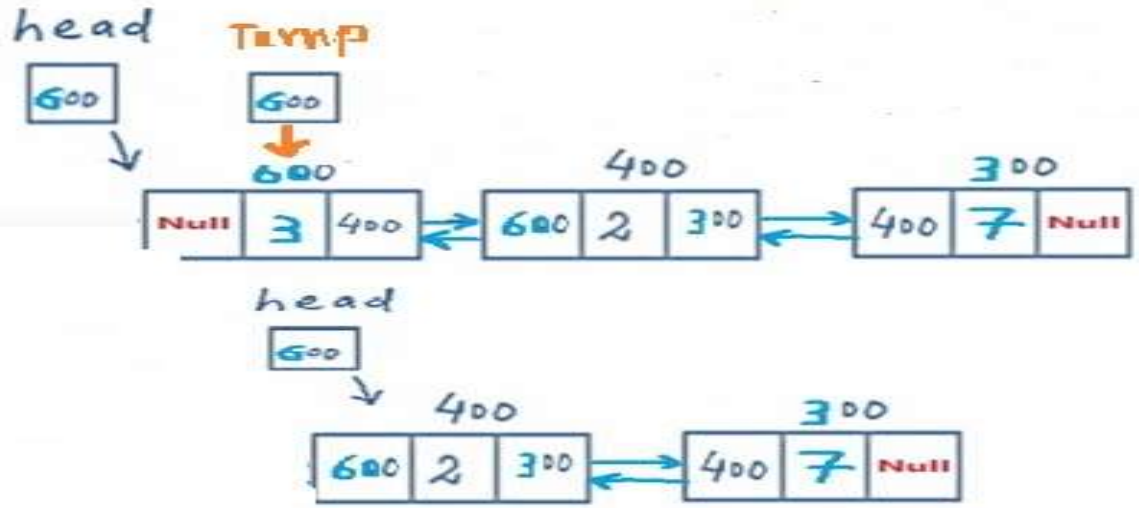



```
Node *ptr = new node()
ptr->data = val;
ptr->Next = Null
```

```
while (Temp->next != Null)
 {
   Temp = Temp> next;
 }
Temp->next = ptr
 Ptr->Pre = Temp
```

Doubly Linked Lists (Delete First)

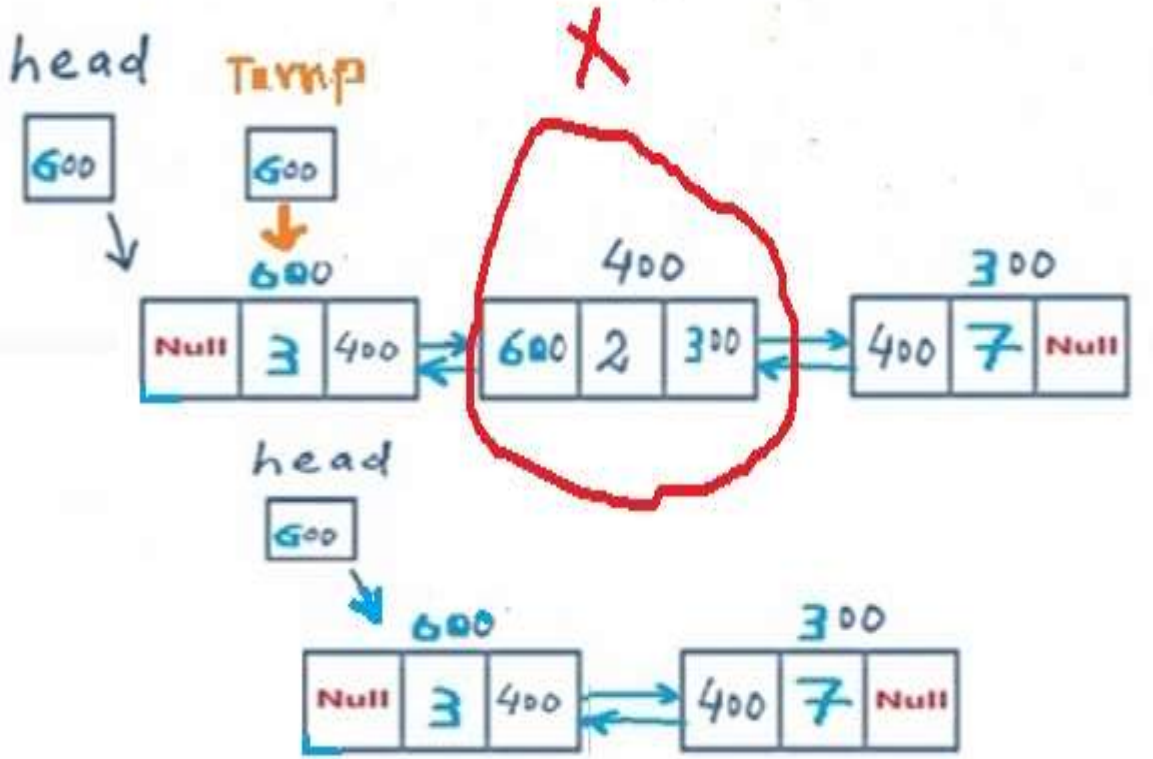
```
struct Node  {
    int data;
    struct Node* next;
    struct Node* prev;
};
```




```
while (Head != Null)
{
   Head = Head> next;
   Head->Prev = Null
   Delete Temp
}
```

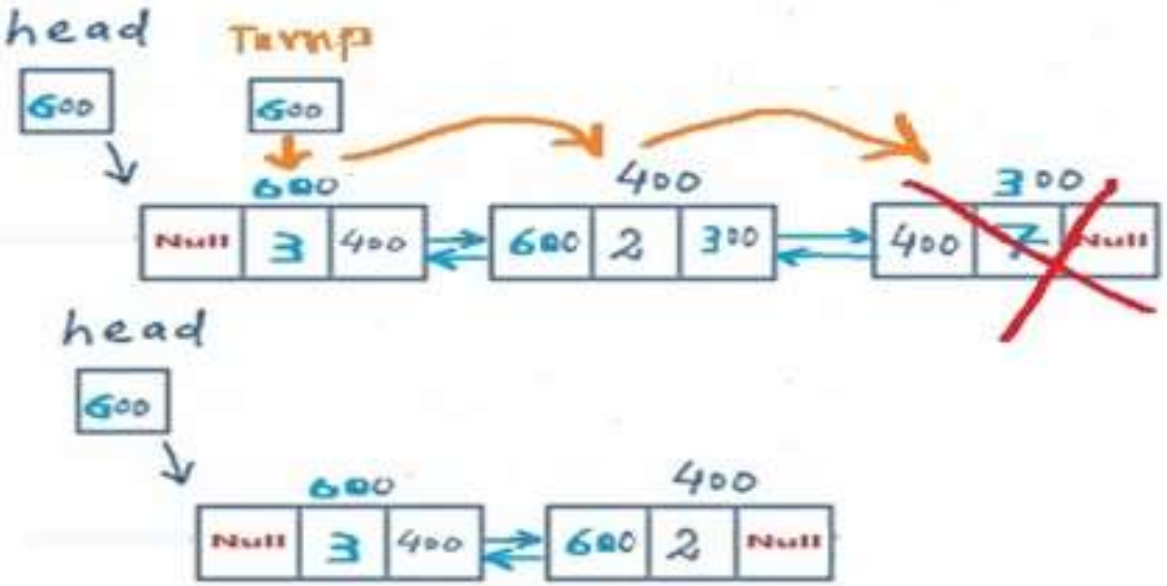
Doubly Linked Lists (Delete Middle Item)

```
struct Node  {
    int data;
    struct Node* next;
    struct Node* prev;
};
```



Doubly Linked Lists (Delete Last)

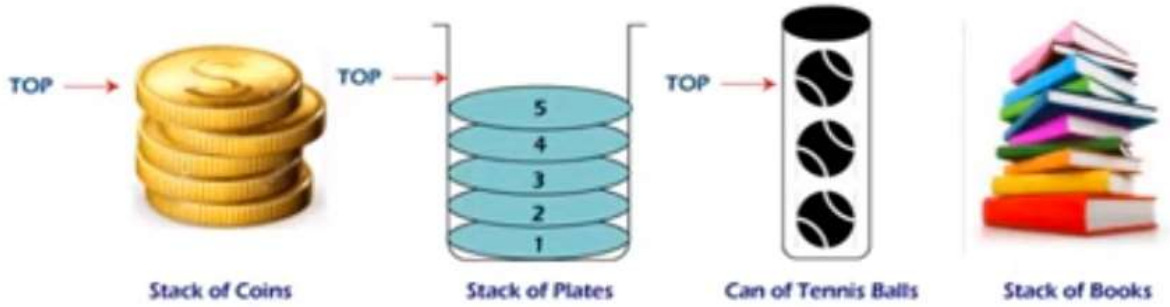
```
struct Node  {
    int data;
    struct Node* next;
    struct Node* prev;
};
```

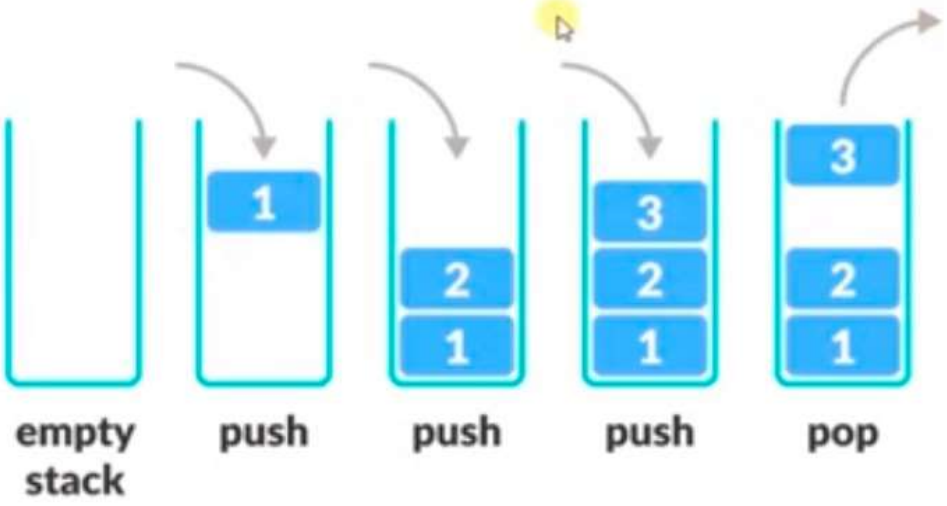



```
while (Temp->Next != Null)
  {
Temp = Temp> next;
   }
Temp->Pre = Null
Delete Temp
```

Abstract Data Types (ADT)

Stack – Last in First Out - LIFO

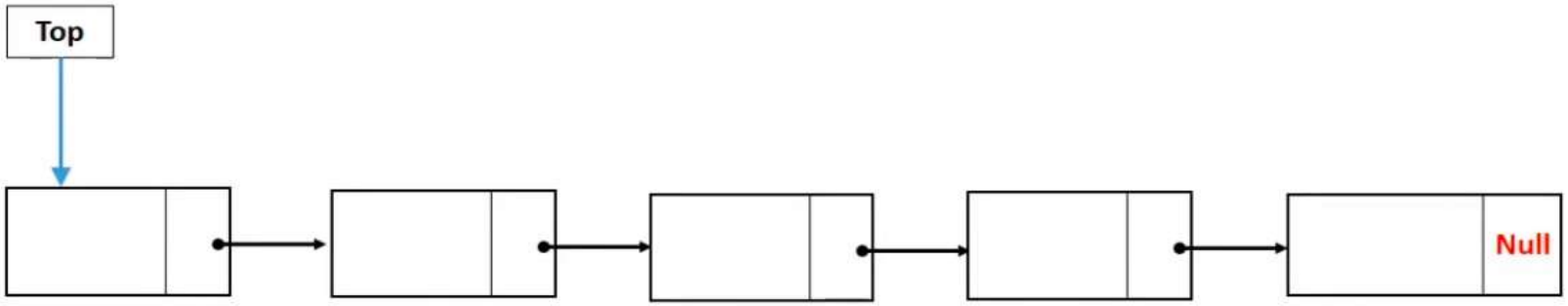




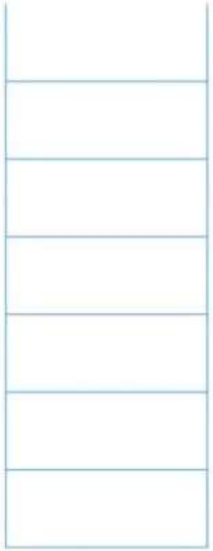
Stack Operations

- push( ) : Insert the element into the top of Stack.
- Pop ( ) : Return top element from the Stack.
- Peek ( ): Return the top element.
- Display ( ): Print all element of Stack.
- isEmpty ( ) : check whether the stack is empty or not
- isFull ( ) : check if there is a place to add an extra item or not.
- Count ( ) : return the count of stack items.
- Search ( ) : search for a specific item in the stack.

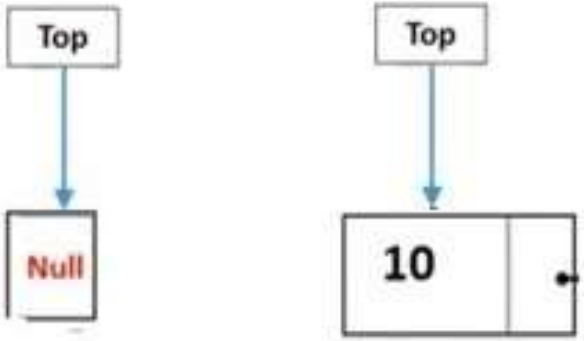
Stack Operations (push)



10 20 30 40 50



Stack Operations (push)



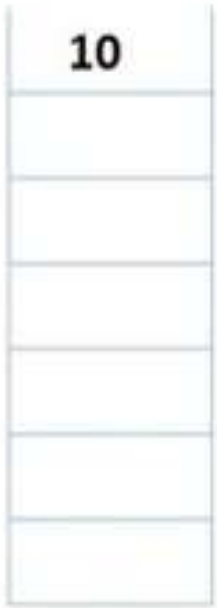
Top = Null

Push (10)

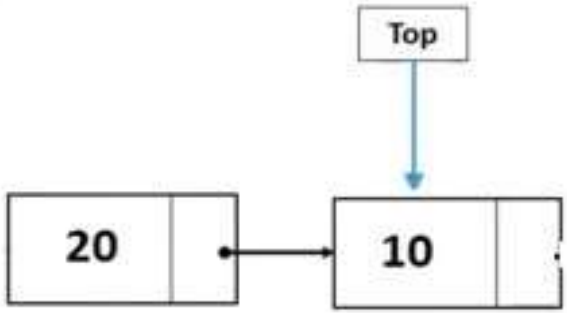
```
Node *Newnode
Newnode->Data =10;
Newnode->next=Null
```

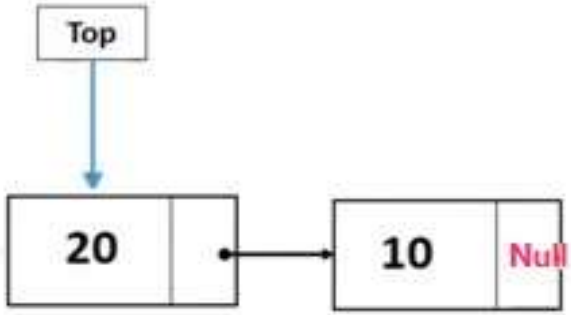
```
If (Top == Null)
{
Cout<<"Stack is empty";
}
```

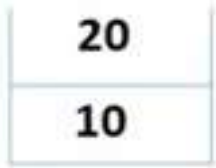
```
Top =Null
Top = Newnode;
```



Stack Operations (push)

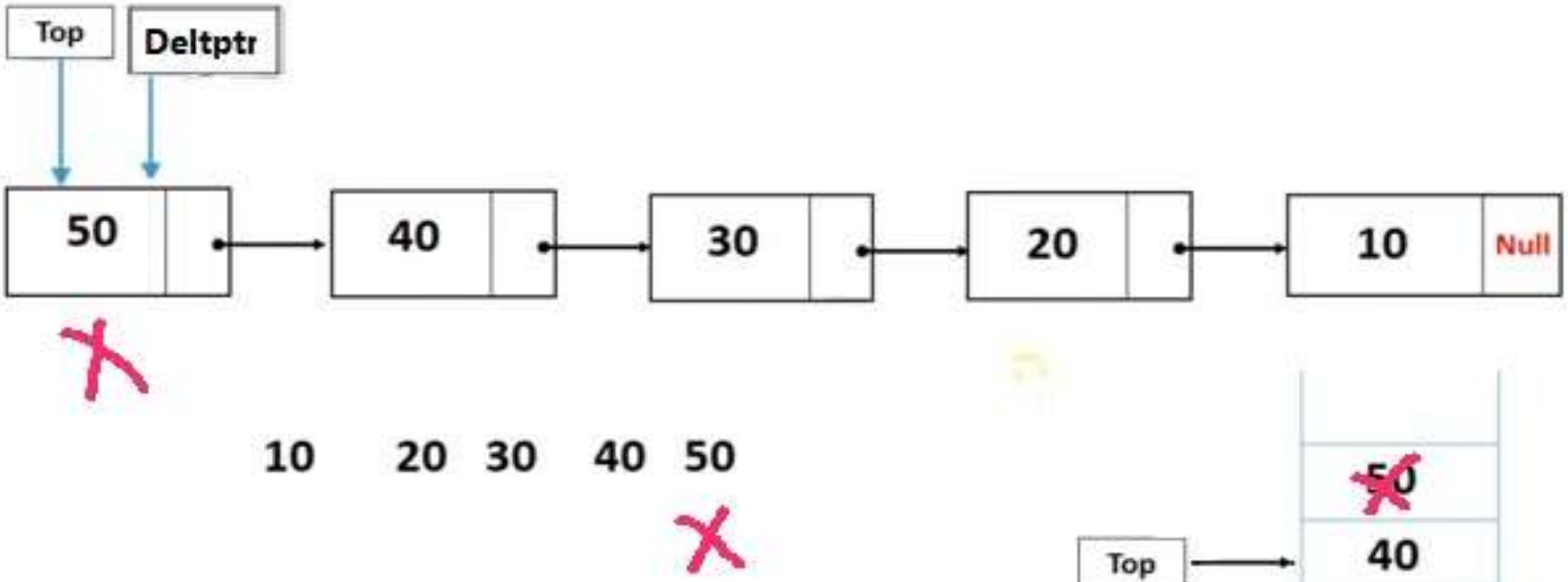






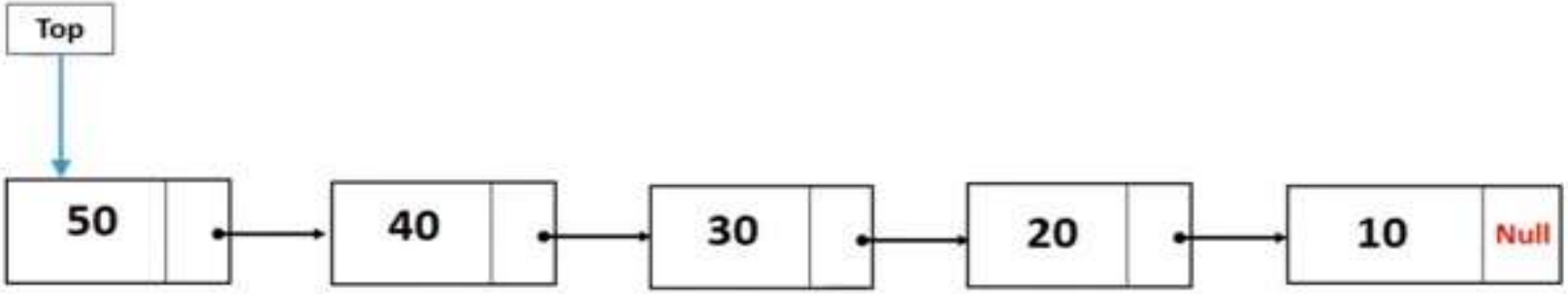

```
Node *Newnode = new Node ()
Newnode->Data = 20;
Newnode-> next = Top;
Top = Newnode;
```

Stack Operations (PoP)



Node *Delptr = Top;
Top= Top->next;
Delete Delptr

Stack Operations (Peek)

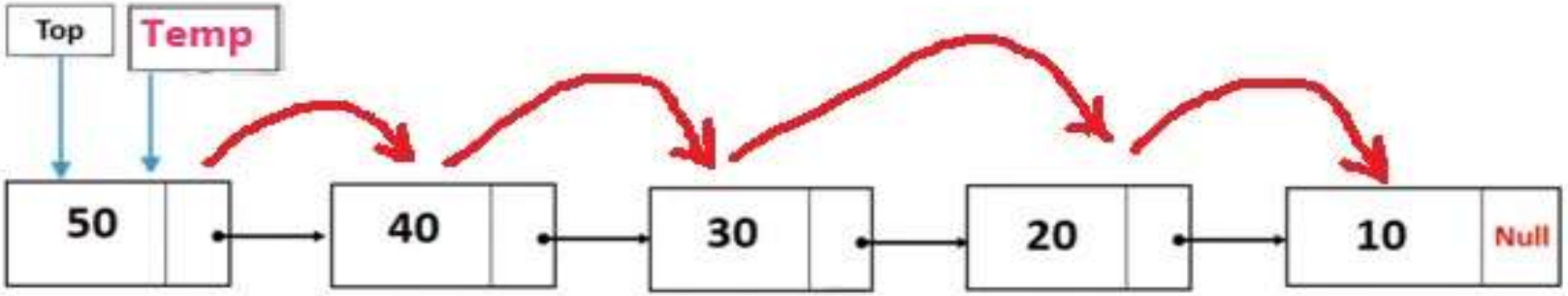


10 20 30 40 50

50
40
30
20
10

Top-> Data

Stack Operations (Display)

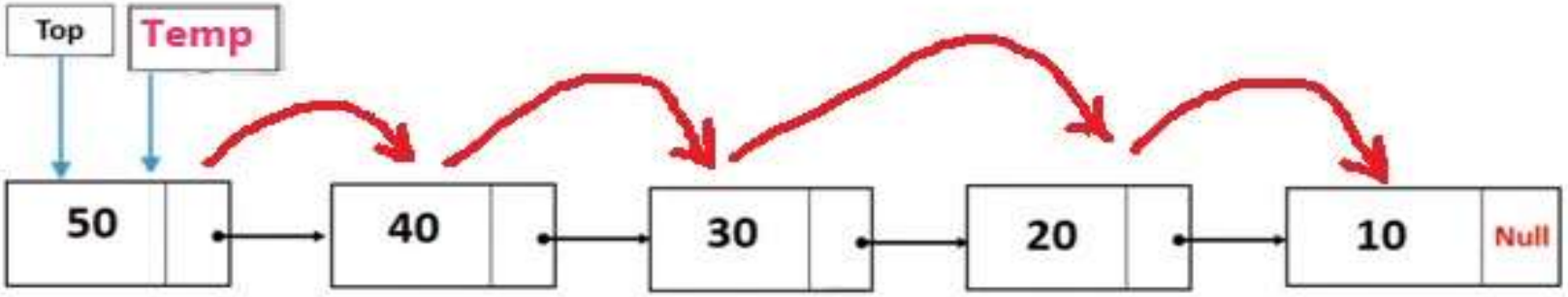


10 20 30 40 50

50
40
30
20
10

```
Node *Temp = Top;
Cout <<("List elements are - \n");
while(Temp->next != NULL) {
 Cout <<(" --->",Temp->data);
  Temp = Temp->next;
}
```

Stack Operations (Count)

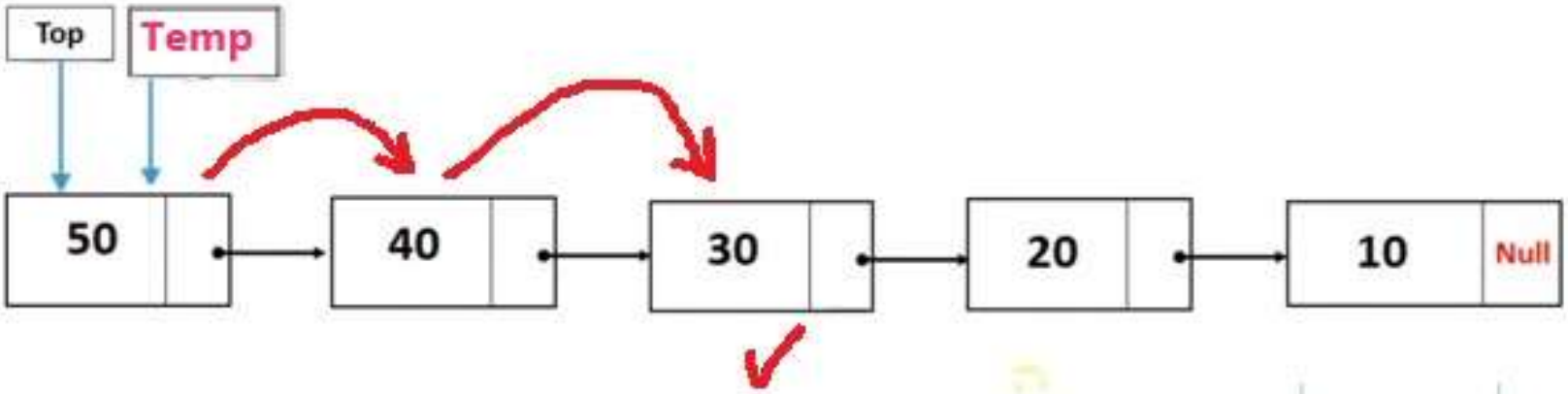


10 20 30 40 50

50
40
30
20
10

```
Int Counter = 0;
Node *Temp = Top;
Cout <<("List elements are - \n");
while(Team->next != NULL)
 {
 Counter ++;
 Cout <<(" --->",Temp->data);
  Temp = Temp->next;
}
Return Counter
```

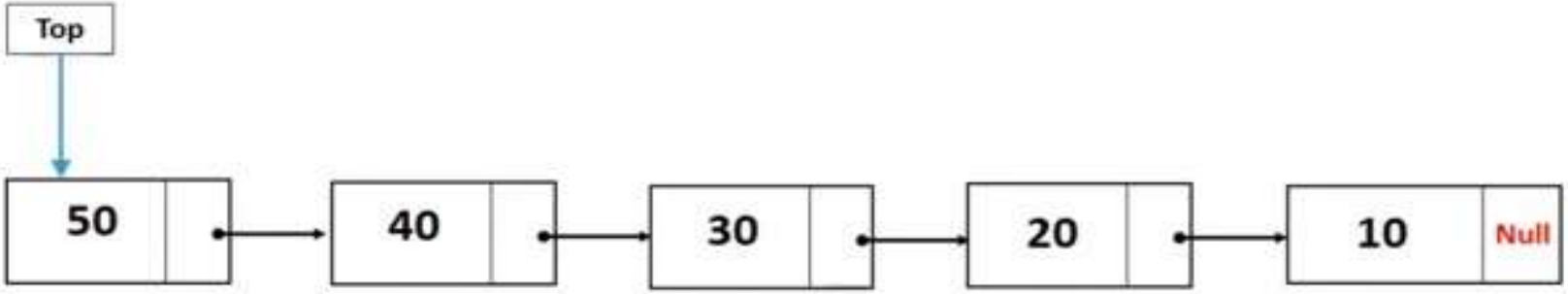
Stack Operations (Search)

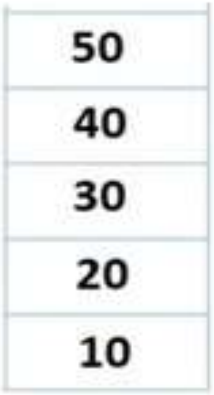


30

```
Node *Temp = Top;
while(Temp != NULL)
 {
If (Temp->Data == Item)
   {
     Cout <<“ item is”, Temp->data;
     }
 Else
   {
     Temp = Temp->Next;
   }
```

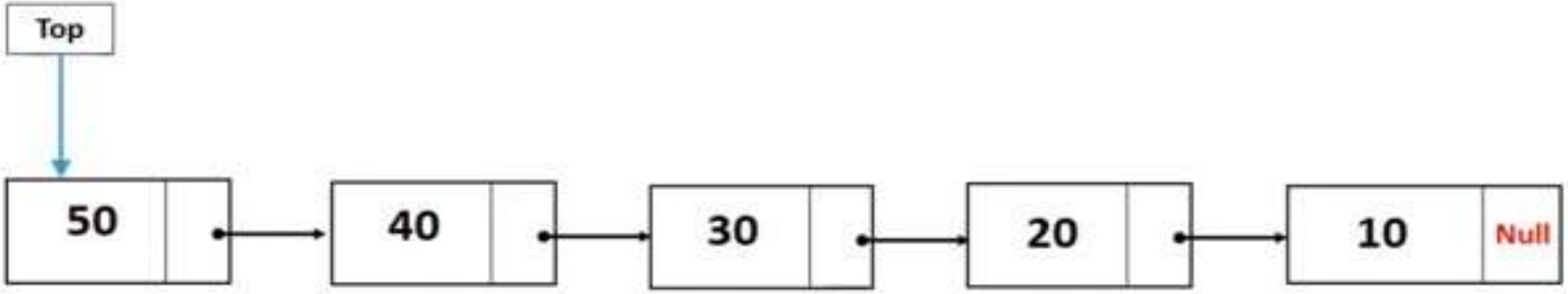
Stack Operations (IsEmpty)






```
If (Top == Null);
Cout<<"there is items";
```

Stack Operations (IsFull)




```
Node *Newnode = new node()
If (Newnode == Null);
Cout<<“there is no place”;
```

50
40
30
20
10